

Design Documentation

Software Engineering - SVNC.00.308

Project: StayEase Hotel Reservation System

Type: Hotel Reservation

Team: Cat Bacillus (Kyrylo Pryiomyshev, Danylo Chernov, Daniil Ustuhov)

Date: February 20, 2026

Version: 1.0

1. Design Overview

1.1 Purpose

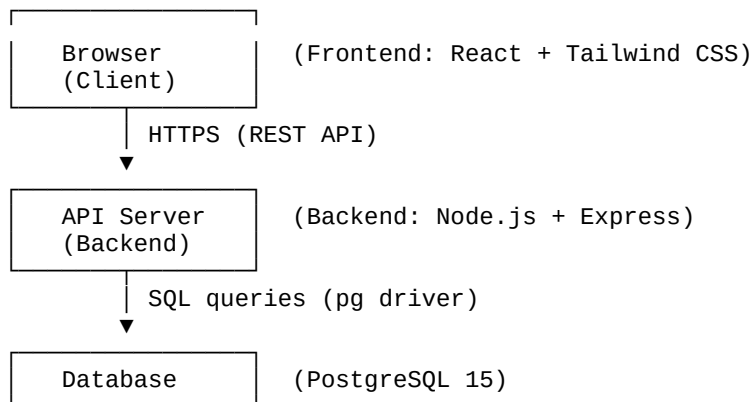
This document describes the system architecture, component structure, data models, and technology decisions for the StayEase Hotel Reservation System. The design follows a three-tier web architecture with a React frontend client, a Node.js/Express backend API server, and a PostgreSQL database layer.

1.2 Design Principles

- **Modularity:** Separate concerns across UI, business logic, and data access layers
- **Scalability:** Design to accommodate growing number of guests and rooms
- **Security:** Protect guest personal data and prevent unauthorized access
- **Usability:** Intuitive interface requiring minimal training for both guests and staff
- **Data Integrity:** Ensure booking consistency through ACID transactions to prevent double bookings

2. System Architecture

2.1 Architecture Diagram



External: Email Service (SendGrid/SMTP) for notifications

2.2 Architecture Style

Our Choice: Three-Tier Client-Server Architecture

Justification: This architecture cleanly separates the presentation layer (React SPA), application layer (Express API), and data layer (PostgreSQL). It is well-suited for web applications with clear layer boundaries, allows independent development of frontend and backend, and provides a standard deployment model. Microservices would add unnecessary complexity for a hotel with a single property.

3. Component Specifications

3.1 Component: Frontend / User Interface

Responsibility: Displays room catalog, handles room search, allows guests to make/modify/cancel reservations, provides staff dashboard for managing check-ins and occupancy.

Key UI Views

- Login / Registration Page: Email, password, name, and phone fields for guests; separate staff login
- Room Search Page: Date picker (check-in/check-out), guest count, room type filter, search results with availability
- Room Details Page: Room photos, amenities, description, price per night, and 'Book Now' button
- Booking Confirmation Page: Reservation summary with dates, room type, total price, special requests field
- My Reservations Page: List of upcoming and past bookings with modify/cancel options
- Staff Dashboard: Today's arrivals/departures, occupancy overview, reservation search, check-in/check-out actions

Interactions with Backend

- Calls REST API endpoints for all data operations (GET /rooms, POST /reservations, etc.)
- Sends JWT token in Authorization header for authenticated requests
- Receives JSON responses and renders UI components accordingly

3.2 Component: Backend API / Application Logic

Responsibility: Handles business logic, validates requests, manages data access, enforces booking rules (no double bookings, cancellation policy), and sends email notifications.

Key Modules

- Authentication Module: Guest/staff login, JWT generation and validation, password hashing with bcrypt
- Room Management Module: CRUD operations for rooms, availability calculation based on date ranges
- Reservation Module: Create/modify/cancel reservations, prevent overbooking via transactional locks, manage room status
- Notification Module: Send email confirmations, modifications, cancellations, and pre-arrival reminders
- Reporting Module: Occupancy rate calculations, reservation statistics by date range

API Endpoints

Method	Endpoint	Description	Request Body	Response
POST	/api/auth/register	Guest	{email, password,	{userId, token}

		registration	name, phone}	
POST	/api/auth/login	User login	{email, password}	{token, role, userId}
GET	/api/rooms	Get room catalog	Query: ?type=deluxe	[[roomId, name, type, price, ...]]
GET	/api/rooms/available	Search available rooms	Query: ?checkIn=...&checkOut=...&guests=2	[[roomId, type, price, available]]
GET	/api/rooms/:id	Get room details	-	{roomId, name, amenities, photos}
POST	/api/reservations	Create reservation	{roomId, checkIn, checkOut, guests, requests}	{reservationId, status}
PUT	/api/reservations/:id	Modify reservation	{checkIn, checkOut, roomId}	{reservationId, status}
DELETE	/api/reservations/:id	Cancel reservation	-	{status: cancelled}
GET	/api/reservations/user/:id	Get user reservations	-	[[reservationId, room, dates, status]]
PUT	/api/reservations/:id/checkin	Check-in guest (staff)	-	{status: checked_in}
PUT	/api/reservations/:id/checkout	Check-out guest (staff)	-	{status: completed}
GET	/api/reports/occupancy	Occupancy report (manager)	Query: ?from=...&to=...	{rate, totalRooms, booked}

3.3 Component: Database / Data Storage

Responsibility: Persist guest accounts, room inventory, reservations, and attendance/check-in records with full ACID compliance.

Database Type: Relational (PostgreSQL 15)

Why This Choice: Highly structured data with clear relationships (guests, rooms, reservations). ACID transactions are critical for preventing double bookings. PostgreSQL offers mature date/range handling, which is essential for reservation date overlap checks.

4. Data Model

4.1 Entity-Relationship Diagram

Users (users)

- user_id (PK, SERIAL)
- email (UNIQUE, NOT NULL)
- password_hash (NOT NULL)
- full_name (NOT NULL)
- phone (NOT NULL)
- role (guest | staff | manager)
- created_at (TIMESTAMP)

Rooms (rooms)

- room_id (PK, SERIAL)
- room_number (UNIQUE, NOT NULL)
- type (standard | deluxe | suite | family)
- capacity (INT, NOT NULL)
- price_per_night (DECIMAL, NOT NULL)
- description (TEXT)
- amenities (TEXT[])
- status (available | occupied | maintenance)

Reservations (reservations)

- reservation_id (PK, SERIAL)
- user_id (FK -> users.user_id)
- room_id (FK -> rooms.room_id)
- check_in_date (DATE, NOT NULL)
- check_out_date (DATE, NOT NULL)
- num_guests (INT, NOT NULL)
- special_requests (TEXT)
- total_price (DECIMAL)
- status (confirmed|cancelled|checked_in|completed)
- created_at (TIMESTAMP)

4.2 Entity Details

Entity: Users

Attribute	Type	Constraints	Description
user_id	SERIAL	PRIMARY KEY	Unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	User email for login
password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
full_name	VARCHAR(150)	NOT NULL	Guest or staff full name
phone	VARCHAR(20)	NOT NULL	Contact phone number
role	VARCHAR(20)	NOT NULL, DEFAULT 'guest'	User role: guest, staff, manager
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

Entity: Rooms

Attribute	Type	Constraints	Description
room_id	SERIAL	PRIMARY KEY	Unique room identifier
room_number	VARCHAR(10)	UNIQUE, NOT NULL	Physical room number (e.g., 101)
type	VARCHAR(20)	NOT NULL	Room type: standard, deluxe, suite, family
capacity	INT	NOT NULL, CHECK > 0	Maximum guests
price_per_night	DECIMAL(10,2)	NOT NULL	Nightly rate in EUR
description	TEXT	-	Room description and features
amenities	TEXT[]	-	Array of amenity tags
status	VARCHAR(20)	DEFAULT 'available'	Current room status

Entity: Reservations

Attribute	Type	Constraints	Description
reservation_id	SERIAL	PRIMARY KEY	Unique reservation ID
user_id	INT	FK -> users.user_id	Guest who booked
room_id	INT	FK -> rooms.room_id	Room reserved
check_in_date	DATE	NOT NULL	Arrival date
check_out_date	DATE	NOT NULL, > check_in	Departure date
num_guests	INT	NOT NULL, CHECK > 0	Number of guests
special_requests	TEXT	NULLABLE	Guest special requests
total_price	DECIMAL(10,2)	NOT NULL	Total cost (nights x rate)
status	VARCHAR(20)	DEFAULT 'confirmed'	Booking status
created_at	TIMESTAMP	DEFAULT NOW()	Booking creation time

5. Technology Stack

Layer	Technology	Justification
Frontend	React 18 + Tailwind CSS	Modern component-based UI, rapid styling with utility classes, large ecosystem
Backend	Node.js 20 + Express.js	JavaScript full-stack consistency, fast async I/O, extensive middleware ecosystem
Database	PostgreSQL 15	ACID-compliant, excellent date/range handling, mature and reliable for transactional data
ORM / Query	pg (node-postgres)	Lightweight, direct SQL control, better performance than full ORM for complex date queries
Authentication	JWT + bcrypt	Stateless token-based auth, industry-standard password hashing
Email	SendGrid API	Reliable transactional email delivery, free tier sufficient for hotel volume
Hosting	DigitalOcean Droplet	Cost-effective VPS hosting, simple deployment, EU datacenter available

6. Security Considerations

6.1 Authentication and Authorization

- Passwords hashed with bcrypt (cost factor 12, ~250ms per hash)
- JWT tokens for stateless session management, 24-hour expiry with refresh mechanism
- Role-based access control: guests can only access their own data; staff can manage reservations; manager can access reports

6.2 Data Protection

- HTTPS (TLS 1.2+) for all client-server communication
- SQL injection prevention via parameterized queries (node-postgres \$1, \$2 placeholders)
- Input validation and sanitization on all user inputs (express-validator)
- Rate limiting on authentication endpoints (100 requests per 15 min per IP)

6.3 Privacy

- Guest personal data (name, email, phone) accessible only to authorized staff and the guest themselves
- Staff cannot view guest passwords (hashed and inaccessible)
- Database access restricted to application service account only

7. Interface Specifications

7.1 User Interface Mockups

Screen 1: Login Page

- Fields: Email, Password
- Buttons: Login, Register, Forgot Password
- Behavior: On success, redirect to room search (guest) or dashboard (staff)

Screen 2: Room Search and Results

- Inputs: Check-in date, Check-out date, Number of guests, Room type filter (optional)
- Results: Card grid showing room type, photo, price/night, capacity, amenities, availability status
- Actions: 'Book Now' button on each available room card

Screen 3: Booking Confirmation

- Display: Room details, selected dates, number of nights, total price calculation
- Inputs: Special requests (textarea), estimated arrival time
- Actions: 'Confirm Booking' button, 'Back to Search' link

Screen 4: My Reservations

- Display: List of reservations sorted by date (upcoming first)
- Each card shows: Room type, check-in/out dates, total price, status badge
- Actions: 'Modify' and 'Cancel' buttons (cancel disabled within 24h of check-in)

Screen 5: Staff Dashboard

- Sections: Today's Check-ins, Today's Check-outs, Occupancy Summary, Reservation Search
- Actions: Mark Check-in, Mark Check-out buttons per reservation

8. Performance and Scalability

8.1 Expected Load

- Rooms: 50 rooms across 4 types (standard, deluxe, suite, family)
- Users: 500 registered guests, 5 staff members, 1 manager
- Concurrent Users: Up to 100 during peak booking periods
- Data Volume: ~200 reservations per month, 12-month retention

8.2 Performance Targets

- Room search results: Under 2 seconds
- Reservation creation: Under 3 seconds (including availability check)
- Dashboard page load: Under 2 seconds
- Database queries optimized with indexes on `room_id`, `user_id`, `check_in_date`, `check_out_date`

8.3 Scalability Strategy

- Vertical scaling: Upgrade server resources as traffic grows
- Database connection pooling (pg-pool, max 20 connections)
- Caching room catalog data (in-memory, refreshed every 5 minutes)
- CDN for static frontend assets (images, CSS, JS bundles)

9. Design Trade-Offs

Trade-Off 1: Monolith vs. Microservices

Decision: Three-tier monolith

Rationale: Single-property hotel with a 3-person team. Microservices would add deployment complexity, inter-service communication overhead, and distributed debugging challenges without proportional benefits at this scale.

Trade-Off 2: SQL vs. NoSQL Database

Decision: Relational database (PostgreSQL)

Rationale: Reservation data is highly structured with clear entity relationships. ACID transactions are critical for preventing double bookings. PostgreSQL's date range operators and exclusion constraints make overlap checking efficient and reliable.

Trade-Off 3: Server-Side Rendering vs. SPA

Decision: Single Page Application (React SPA)

Rationale: SPA provides smoother user experience for the search-and-book workflow. SEO is not a priority since this is an internal hotel tool, not a public listing. SPA reduces server load by offloading rendering to the client browser.

10. Sign-Off

Prepared By: Cat Bacillus (Kyrylo Pryiomyshev, Danylo Chernov, Daniil Ustuhov)

Reviewed By: Elena Petrova (Hotel Manager) / Nicolai Morozov (Instructor)

Approval Date: February 23, 2026

Design Lead Signature: _____ Date: _____